

Empirical Analysis for “Predictive conditional alphas” (Ditaso et. al 2026)

Paper authors: Valentina Corradi, Walter Distaso, Marcelo Fernandes, and Asger Lunde

9 Jul 2026

Table of contents

1	Empirical Analysis for “Predictive conditional alphas” (Ditaso et. al 2026)	3
1.1	Repository organization	3
2	Ditaso (2026) - Data processing: factors	5
2.1	About the data and methodology	5
2.2	Code setup	6
2.3	Importing and dates	6
2.4	Log-returns and 5-minute aggregation	8
2.5	Exporting	10
3	Ditaso (2026) - Data processing: stocks	11
3.1	About the data and methodology	11
3.2	Current methodology	12
3.3	Code setup	13
3.4	Quality checks	13
3.4.1	River graphs	13
3.5	Raw data download	15
4	Factor PCA	19
5	Realized betas and risk-adjusted returns	20
6	Conditional alphas	21

1 Empirical Analysis for “Predictive conditional alphas” (Ditso et al. 2026)

This project contains the empirical analysis for the paper “Predictive conditional alphas” by Ditso et al. (2026). The relevant version of the paper can be found [here](#).

The general steps of the empirical analysis are:

- Download and process factor and stock data.
- Extract high-frequency principal components from the factors.
- Estimate realized betas from the stock returns and the principal components.
- Estimate conditional alphas from the risk-adjusted returns.

The detailed methodology for each step is described in the associated report chapter or `.ipynb` notebook in [src/](#).

This project has a repository and a PDF report associated. Both are explained in this text.

1.1 Repository organization

The repository is organized as follows:

- [data](#):
 - [data/stocks_1min_returns](#) and [data/stocks_5min_returns](#): individual stock log-returns at the 1-minute and 5-minute frequencies. Stocks are ID'd by their CRSP PERMNO. Only some stocks are committed. The raw data [data/trades.db](#) is not committed and is from the TAQ database.
 - [data/ff6_1min_returns_raw.csv](#), [data/ff6_1min_returns.csv](#) and [data/ff6_5min_returns.csv](#): Factor returns at the 1-minute and 5-minute frequencies. The raw data is not committed and available at www.sakethaleti.com/data.
- [src](#):
 - [src/data_factors.ipynb](#) [src/data_stocks.ipynb](#): Data checking and processing.
 - [src/pca.ipynb](#): High-frequency PCA of the factors estimation.
 - [src/betas.ipynb](#): Realized betas and risk-adjusted returns estimation.
 - [src/alphas.ipynb](#): Conditional alphas estimation.

- `src/parameters.py`, `src/utls.py`: Utility scripts.
- `bibliography`: reference papers.
- `docs`: report of this empirical analysis.
- Configuration files:
 - Docs and repository: `_quarto.yml` `README.md` `.gitignore`
 - Python (`uv` environment): `.python-version` `pyproject.toml` `uv.lock`

2 Ditaso (2026) - Data processing: factors

This notebook mines and processes the factor data needed. Read the [README.md](#) file for context about the project.

2.1 About the data and methodology

Sources:

- Fama-French 6 factors from saketthaleti.com/data, or [this Google Drive link](#). Already at the 1-minute level.

Filters:

- Sample period (same as stocks):
 - Range: 2001-01-02 to 2017-12-29. Stocks that are listed or delisted during this period are included.
 - Hours: regular trading hours (09:30 - 16:00 ET).
 - Days: regular trading days, removing days with market-wide halts and diminished trading hours.

Quality checks:

- Data is very trusted. There should be data for all and only moments the market is open. Specially: there shouldn't be data for a stock and not for the factors; and there should be all 390 minutes of a day with data.
- Assert that the data is in returns (not log).
- Plot the number of observations per day across time, listing the days with less than 390.

Processing steps:

- Download the data into "data/ff6_1min_returns_raw.csv". Alternatively, could be read directly from Google Drive.
- Run the quality checks above.
- Apply the filters above.
- Transform data into log-returns.

- Aggregate the data into 5-minute blocks (09:35, 09:40, ..., 16:00) by summing the returns of the label minute and the 4 minutes before it. Save it into "data/ff6_5min_returns.csv".

2.2 Code setup

```
%reset -f
import importlib
```

```
import os
if os.path.basename(os.getcwd()) == "src": os.chdir("..")

import polars as pl
import numpy as np

import pandas_market_calendars as mcal

import plotnine as gg
from plotnine import ggplot, aes

import src.utils as ut
from src.parameters import PARAMETERS as PARS
importlib.reload(ut)
```

```
<module 'src.utils' from 'c:\\Users\\ricar\\OneDrive\\A-Trabalho\\Freelance\\ra-ditaso2026\\'
```

2.3 Importing and dates

Importing and formatting dates as datetime. First look at the data:

```
data = (
    pl.read_csv("data/ff6_1min_returns_raw.csv")
    .with_columns(
        pl.col("datetime")
        .str.strptime(pl.Datetime, "%Y-%m-%d %H:%M:%S", )
    )
)
cols_fct = ["ff__mkt", "ff__smb", "ff__hml", "ff__rmw", "ff__cma", "ff__umd"]
```

```
print(f"Data shape: {data.shape}")
print("First rows:")
data.head(3)
```

Data shape: (2454420, 7)
First rows:

datetime	ff__mkt	ff__smb	ff__hml	ff__rmw	ff__cma	ff__umd
datetime[s]	f64	f64	f64	f64	f64	f64
1996-01-02 09:30:00	-0.002509	0.001133	-0.000592	-0.000541	-0.000922	-0.003535
1996-01-02 09:31:00	-0.000061	0.000184	0.000053	0.00002	-0.000021	0.000204
1996-01-02 09:32:00	-0.000123	0.000121	-0.000045	0.000123	-0.000098	-0.000083

Checking sorting:

```
if not data["datetime"].is_sorted():
    raise ValueError("Data is not sorted by datetime.")
```

All missing dates are non-trading days in NYSE's calendar (assumed as the same as NASDAQ and AMEX).

```
date_days = data["datetime"].dt.date().unique()
date_gaps = date_days.diff(1, "drop").dt.total_days()

date_miss = []
for day, gap in zip(
    date_days[1:].filter(date_gaps > 1),
    date_gaps.filter(date_gaps > 1)
):
    date_miss.extend(np.arange(day - np.timedelta64(gap - 1, "D"), day).astype("datetime64[D]"))
date_miss = np.array(date_miss, dtype = "datetime64[D]")

date_valid = (
    mcal.get_calendar("NYSE") # Same as NASDAQ and AMEX
    .valid_days(PARS.date_start, PARS.date_end)
    .to_numpy("datetime64[D]")
)

date_miss_valid = date_miss[np.isin(date_miss, np.array(date_valid))]
```

```

if date_miss_valid.size > 0:
    raise ValueError(
        f"There are {len(date_miss_valid)} gaps in days. They begin at:" +
        ut.format_days_byyear(date_miss_valid)
    )

```

Checking non-full days (less than 391 minutes of data). All are special days given by the NYSE's calendar. Will be removed.

```

dates_non_full = PARS.trading_days[
    ~ np.isin(PARS.trading_days, PARS.trading_days_full)
]
print(f"Non-full trading days:" + ut.format_days_byyear(dates_non_full))

data = data.filter(np.isin(data["datetime"].dt.date(), PARS.trading_days_full))

```

Non-full trading days:

- 2001: 07-03, 11-23, 12-24
- 2002: 07-05, 11-29, 12-24
- 2003: 07-03, 11-28, 12-24, 12-26
- 2004: 11-26
- 2005: 11-25
- 2006: 07-03, 11-24
- 2007: 12-24
- 2015: 11-27, 12-24
- 2016: 11-25
- 2017: 07-03, 11-24

2.4 Log-returns and 5-minute aggregation

Data is in returns, and indeed all values are in the range $[-1, 1]$. We can transform it into log-returns by applying the map $r \rightarrow \log(1 + r)$.

```

data_1min = data.with_columns(
    pl.col(cols_fct).log1p()
)

data_1min

```

datetime	ff__mkt	ff__smb	ff__hml	ff__rmw	ff__cma	ff__umd
datetime[s]	f64	f64	f64	f64	f64	f64
2001-01-02 09:30:00	-0.004918	0.00636	-0.001458	-0.002883	-0.000977	-0.011494
2001-01-02 09:31:00	0.000095	-0.00038	-0.000202	0.000268	0.000314	-0.000224
2001-01-02 09:32:00	-0.000312	-0.00004	0.00021	0.000598	0.000524	0.000227
2001-01-02 09:33:00	-0.000307	-0.000065	0.000177	0.000557	0.000068	-0.000511
2001-01-02 09:34:00	-0.001305	0.000494	0.000684	0.000274	0.000593	0.000511
...	...	...	...	...	...	...
2017-12-29 15:56:00	-0.00003	0.00007	-0.000064	0.000003	-0.00009	0.000055
2017-12-29 15:57:00	-0.000033	0.000039	-0.00004	0.000023	-0.000023	0.000103
2017-12-29 15:58:00	-0.000192	-0.000075	-0.000107	0.000056	0.000017	0.000075
2017-12-29 15:59:00	5.8310e-7	-0.000083	-0.000132	-0.000041	0.000018	-0.000061
2017-12-29 16:00:00	-0.001403	0.000706	-0.000426	0.000124	-0.000084	-0.00008

Aggregating below. It is important that the minute blocks are (9:30, 9:35, ...).

```
data_5min = (
    data_1min.groupby_dynamic("datetime", every="5m")
    .agg([pl.col(col).sum() for col in cols_fct])
)

data_5min
```

datetime	ff__mkt	ff__smb	ff__hml	ff__rmw	ff__cma	ff__umd
datetime[s]	f64	f64	f64	f64	f64	f64
2001-01-02 09:30:00	-0.006746	0.006369	-0.000591	-0.001186	0.000522	-0.011491
2001-01-02 09:35:00	-0.000719	-0.001081	0.000788	0.002525	0.000066	-0.000575
2001-01-02 09:40:00	0.00256	-0.002862	-0.002178	0.000718	-0.000589	-0.002095
2001-01-02 09:45:00	-0.000581	-0.000438	-0.000018	-0.000494	0.00174	-0.000943
2001-01-02 09:50:00	-0.000909	-0.000451	0.000601	0.000339	0.002158	-0.000354
...	...	...	...	...	...	...
2017-12-29 15:40:00	-0.000458	-0.000037	-0.00016	0.000045	0.000087	0.000097
2017-12-29 15:45:00	0.000247	-0.000531	0.000275	-0.000324	0.000167	-0.000144
2017-12-29 15:50:00	-0.001065	-0.000379	0.000221	-0.000101	0.000517	-0.000404
2017-12-29 15:55:00	-0.000642	-0.000412	-0.000346	0.000059	-0.000056	0.00015
2017-12-29 16:00:00	-0.001403	0.000706	-0.000426	0.000124	-0.000084	-0.00008

2.5 Exporting

Saving as .csv:

```
data_1min.write_csv("data/ff6_1min_returns.csv")  
data_5min.write_csv("data/ff6_5min_returns.csv")
```

3 Ditaso (2026) - Data processing: stocks

This notebook mines and processes the stock data needed. Read the [README.md](#) file for context about the project.

3.1 About the data and methodology

Sources:

- Intraday tick data from the [NYSE TAQ database](#).
- Securities permanent number (PERMNO) from the [CRSP database](#).

Filters:

- Exchanges: NYSE, NYSE-Arca, NASDAQ, and AMEX (code shares 10 and 11 at the CRSP database).
- Types of trades: regular trades (`TR_CORR = '00'`).
- Sample period(same as factors):
 - Range: 2001-01-02 to 2017-12-29. Stocks that are listed or delisted during this period are included.
 - Hours: regular trading hours (09:30 - 16:00 ET).
 - Days: regular trading days, removing days with market-wide halts and diminished trading hours.
- Minimal liquidity: at most 2.5% of the trading days (excluding pre-listing and post-delisting days) have less than 80% of blocks with no trades. Done separately for both 1 and 5-minute interval blocks.

Quality checks:

- Data is very trusted. Some sanity checks include: no negative or zero prices. no data on non-trading days. data ordered by datetime.
- For each block size, plot a river graph of the number of blocks with data for each day (across some stocks).

Processing steps:

- For each PERMNO:
 - Join all the tickers that belong to the same PERMNO.
 - Go to next PERMNO if data doesn't pass the filters above, else continue.
 - For each 1-minute block t :
 - * Calculate $r_t^1 = \ln(p_t/p_{t-1})$, where p_t is the last seen price at t .
 - * Exclude $p_{9:30}$ to remove overnight effects.
 - * Save the csv `{"datetime": t, "return": r^1_t, "ticker": ticker}` to `"data/stock_returns_1min/perm{PERMNO}.scv"`.
 - For each 5-minute block t (09:35, 09:40, ..., 16:00):
 - * Calculate $r_t^5 = \sum_{i=0}^4 r_{t-i}^1$.
 - * Save the csv `{"datetime": t, "return": r^1_t, "ticker": ticker}` to `"data/stock_returns_1min/perm{PERMNO}.scv"`.

This way, all 390 1-minute blocks and all 78 5-minute blocks have data, even if it is zero return. We should also save the number of trades that happened in each block, in a column `"n_obs_block"`.

Note that the “last seen price” is not the undisputed correct price to use for calculating returns. Other methods include the midpoint of the bid-ask spread and the volume-weighted average price.

3.2 Current methodology

The `"data/stock_returns_1min/perm{PERMNO}.scv"` (and 5-min) files were already created in a previous moment, and are available at [this Dropbox link](#). But note that:

- The sanity checks and filtering were not necessarily done. Importantly, days with market-wide halts and diminished trading hours were not removed.
- The 1-minute data is not filled with zeroes in no-trade blocks, but the 5-minute data is. The former doesn't have a `"n_obs_block"` column. The former has, but it counts the number of minutes with data in the 5-minute block.

Remaining steps:

- Download the Dropbox data into `"data/stocks_5min_returns/"` and `"data/stocks_1min_returns/"`.
- Run the quality checks above.
- Apply the filters above.

3.3 Code setup

```
%reset -f
import importlib
```

```
import os
if os.path.basename(os.getcwd()) == "src": os.chdir("..")

import polars as pl
import numpy as np

import plotnine as gg
from plotnine import ggplot, aes

import src.utils as ut
from src.parameters import PARAMETERS as PARS
importlib.reload(ut)
```

```
<module 'src.utils' from 'c:\\Users\\ricar\\OneDrive\\A-Trabalho\\Freelance\\ra-ditaso2026\\
```

3.4 Quality checks

3.4.1 River graphs

Below is calculated the number of minutes with data for each day, for each stock; then a dataframe with the median and quantiles across stocks; then a river graph is plotted.

This function errors if any date found outside `PARAMETERS.trading_days`, guaranteeing that all stocks already conform to the sample window.

```
obs = ut.obs_per_stock_day(100)
```

Processing file 1/50: data/stocks_1min_returns\perm92493.csv

```
cols = {"day": PARS.trading_days, "median": np.nanmedian(obs, axis = 0)}
cols.update({
    f"q{str(q)[-1]}": np.nanquantile(obs, q = q, axis = 0)
    for q in [0.4, 0.6, 0.3, 0.7, 0.2, 0.8, 0.1, 0.9]
})
data = pl.DataFrame(cols)
```

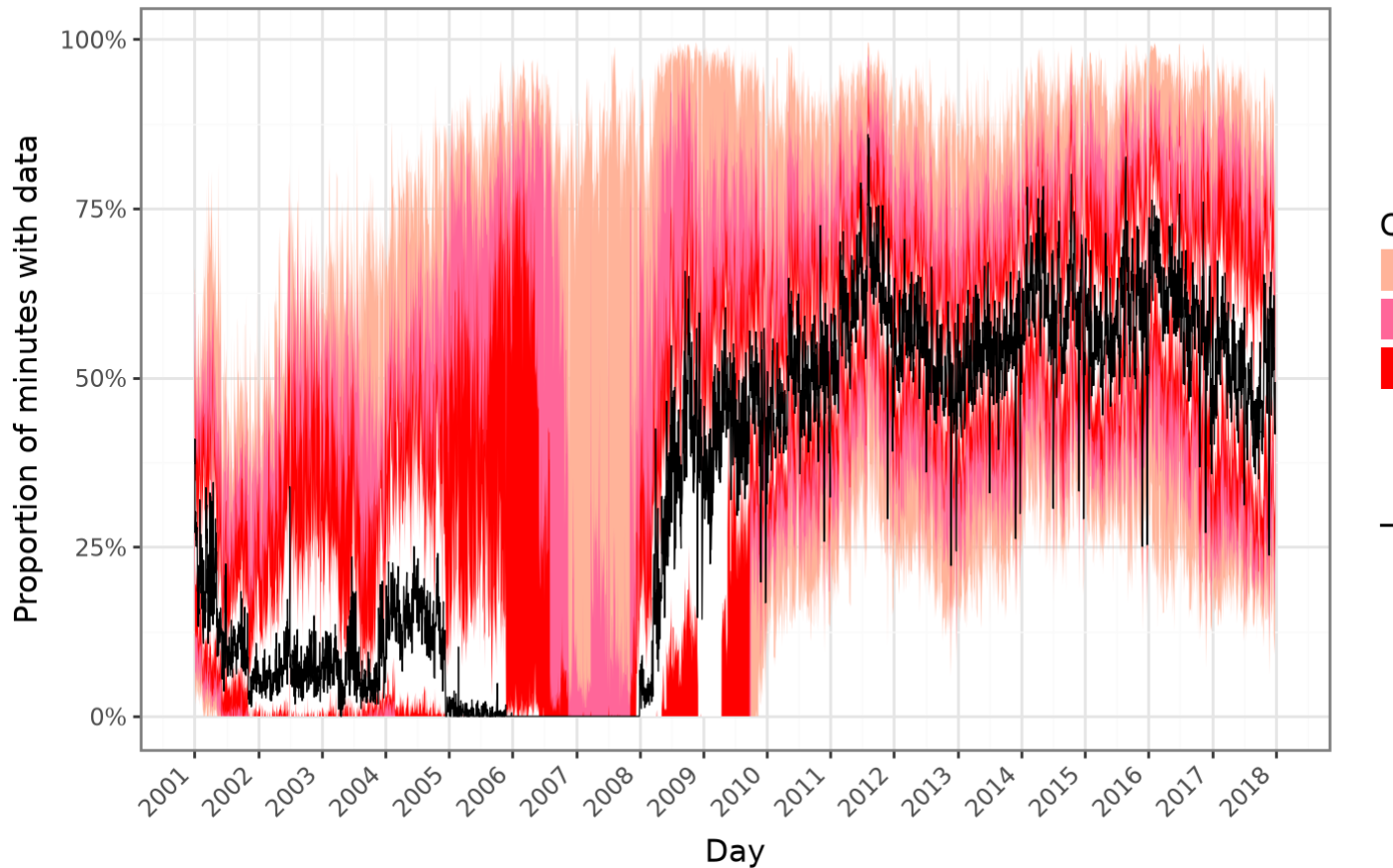
```

fill_colors = {
    "10% - 90%": "#ffb399",
    "20% - 80%": "#ff6699",
    "30% - 70%": "#ff0000",
    "40% - 60%": "#ffffff",
    "median": "black"
}

(
    ggplot(pl.DataFrame(cols), aes(x = "day", group = 1)) +
    gg.geom_ribbon(aes(ymin = "q1", ymax = "q9", fill = "'10% - 90%'")) +
    gg.geom_ribbon(aes(ymin = "q2", ymax = "q8", fill = "'20% - 80%'")) +
    gg.geom_ribbon(aes(ymin = "q3", ymax = "q7", fill = "'30% - 70%'")) +
    gg.geom_ribbon(aes(ymin = "q4", ymax = "q6", fill = "'40% - 60%'")) +
    gg.geom_line(aes(y = "median", color = "'median'"), size = 0.25) +
    gg.scale_x_date(date_labels = "%Y", date_breaks = "1 year") +
    gg.scale_fill_manual(values = fill_colors) +
    gg.scale_color_manual(values = fill_colors) +
    gg.scale_y_continuous(labels = lambda bs: ["{: .0%}".format(b) for b in bs]) +
    gg.labs(
        title = "Proportion of minutes with data within each day",
        subtitle = "Across 100 randomly sampled stocks",
        y = "Proportion of minutes with data", x = "Day",
        fill = "Quantiles", color = " "
    ) +
    gg.theme_bw() +
    gg.theme(
        axis_text_x = gg.element_text(rotation = 45, hjust = 1),
        figure_size = (8, 5)
    )
)

```

Proportion of minutes with data within each day
Across 100 randomly sampled stocks



One could also do the same for the number of 5-minute blocks with data. This should be similar but overall with higher proportions.

3.5 Raw data download

Downloading the big `trades.db` file can be hard. The code below

```
import sqlite3
import requests
import time
import os

class BadHeadersError(Exception):
```

```

pass

# Parameters:
PATH_SAVE = "data/trades.db"
URL = (
    "https://www.dropbox.com/scl/fo/p99wtwn7c5wqamwb348fj/AAHB_eGp3u3KC-j0Kadfqnw/"
    "factor_neutralized_returns_DELIVERY/data/raw/trades.db"
    "?rlkey=hv7qnw3j2utr0pokpw25bs7kr&dl=1"
)

DATA_ETAG = "1780314867411941d" # Dropbox id for file version
DATA_SIZE = 61940215808 # 57GB
# Both hard coded from earlier request

CHUNK_SIZE = 8 * 1024 ** 2 # 8MB
MAX_RETRIES = 20

# Initializing session:
headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,
    "Accept-Language": "pt-BR,pt;q=0.9,en-US;q=0.8,en;q=0.7,es-ES;q=0.6,es;q=0.5",
    "Upgrade-Insecure-Requests": "1",
    "DNT": "1",
    "sec-ch-ua": '"Google Chrome";v="149", "Chromium";v="149", "Not)A;Brand";v="24"',
    "sec-ch-ua-mobile": "?0",
    "sec-ch-ua-platform": '"Windows"',
}
# Could add `Connection: keep-alive` and `Keep-Alive: timeout=60, max=1000`,
# but probably won't do much
s = requests.Session()
s.headers.update(headers)

# Main loop:
print(f"Starting download at {time.strftime('%Y-%m-%d %H:%M:%S')}")
data_got = 0
iteration = 0
while data_got < DATA_SIZE and iteration < MAX_RETRIES:
    iteration += 1
    time.sleep(1.25 ** (iteration + 5))

```

```

print(
    f"{iteration}. Progress: {data_got / (1024**3):.2f} GB / "
    f"{DATA_SIZE / (1024**3):.2f} GB ({data_got / DATA_SIZE:.1%}) ..."
)

if os.path.exists(PATH_SAVE):
    data_got = os.path.getsize(PATH_SAVE)
s.headers.update({"Range": f"bytes={data_got}-"})

try:
    with s.get(URL, stream = True) as response:
        response.raise_for_status()

        etag = response.headers.get("Etag", "").replace("'", '')
        if etag and etag != DATA_ETAG:
            print(
                f"Warning: Etag mismatch. Expected {DATA_ETAG}, got {etag}."
                "Continuing download..."
            )

        content_type = response.headers.get("Content-Type", "").lower()
        if not content_type is 'application/binary':
            raise BadHeadersError("Response is not a binary file.")

        content_length = int(response.headers.get("Content-Length", 0))
        if content_length and content_length != DATA_SIZE - data_got:
            raise BadHeadersError(
                f"Content-Length mismatch. Expected {DATA_SIZE - data_got},"
                f" got {content_length}."
            )

        with open(PATH_SAVE, "ab") as f:
            for chunk in response.iter_content(chunk_size = CHUNK_SIZE):
                if chunk: # Filter out keep-alive empty chunks
                    f.write(chunk)
except requests.exceptions.RequestException as e:
    print(f"\nNetwork drop encountered: {e}. Retrying connection.")
    continue
except BadHeadersError as e:
    print(f"\n{e}. Retrying connection.")
    continue
finally:

```

```
s.close()

try:
    print("Download complete! Checking database integrity...")
    conn = sqlite3.connect(PATH_SAVE)
    cursor = conn.cursor()
    cursor.execute("SELECT name FROM sqlite_master WHERE type='table';")
    tables = cursor.fetchall()
    conn.close()
except sqlite3.Error as e:
    print(f"SQLite database error: {e}")
```

4 Factor PCA

To be created...

5 Realized betas and risk-adjusted returns

To be created...

6 Conditional alphas

To be created...